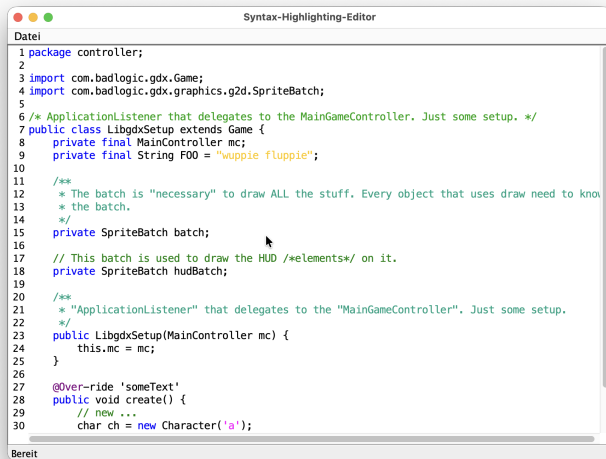


Einführung in ANTLR

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Motivation & Einordnung

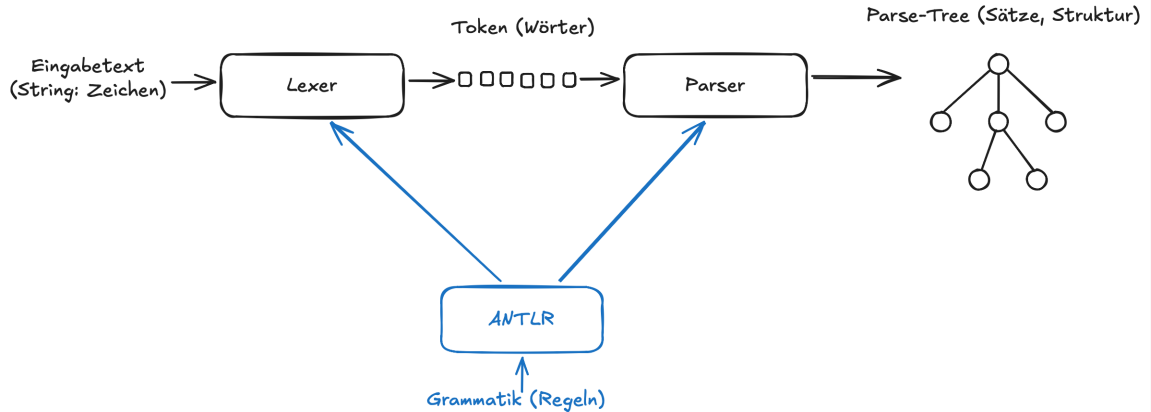


The image shows a window titled "Syntax-Highlighting-Editor" with a "Datei" menu. The code is Java code for a game controller, featuring syntax highlighting for keywords, comments, and strings. The code is as follows:

```
1 package controller;
2
3 import com.badlogic.gdx.Game;
4 import com.badlogic.gdx.graphics.g2d.SpriteBatch;
5
6 /* ApplicationListener that delegates to the MainGameController. Just some setup. */
7 public class LibgdxSetup extends Game {
8     private final MainController mc;
9     private final String F00 = "wuppie fluppie";
10
11     /**
12      * The batch is "necessary" to draw ALL the stuff. Every object that uses draw need to know
13      * the batch.
14      */
15     private SpriteBatch batch;
16
17     // This batch is used to draw the HUD /*elements*/ on it.
18     private SpriteBatch hudBatch;
19
20     /**
21      * "ApplicationListener" that delegates to the "MainGameController". Just some setup.
22      */
23     public LibgdxSetup(MainController mc) {
24         this.mc = mc;
25     }
26
27     @Override 'someText'
28     public void create() {
29         // new ...
30         char ch = new Character('a');
```

Von regulären Ausdrücken zu “richtigen” Bäumen: ANTLR

ANTLR als Blackbox-Pipeline



Technische Einbindung (Gradle, Projektstruktur)

```
plugins {  
    id 'java'  
    id 'antlr'  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    antlr 'org.antlr:antlr4:4.13.2'  
    implementation 'org.antlr:antlr4-runtime:4.13.2'  
}
```

Beispielgrammatik

```
grammar MiniCalc;

// Programm besteht aus beliebig vielen Statements
prog  : stmt* EOF ;

// Statement: entweder Zuweisung oder nackter Ausdruck
stmt  : ID '=' expr ';'
      | expr ';'
      ;

// Ausdruck: eine oder mehrere Zahlen, addiert
expr  : INT ('+' INT)* ;

// LEXER-Regeln (Token)
ID    : [a-z][a-zA-Z0-9_]* ;
INT   : [0-9]+ ;

WS    : [ \t\r\n]+ -> skip ;
```

Minimaler Java-Code: Zerlegen der Eingabe in Token

```
var input = CharStreams.fromString(text);
var lexer = new MiniCalcLexer(input);
var tokens = new CommonTokenStream(lexer);

tokens.fill(); // fill stream (fetch all tokens from lexer)

for (var t : tokens.getTokens()) {
    var tokenName = MiniCalcLexer.VOCABULARY.getSymbolicName(t.getType());
    System.out.printf(
        "%-10s line=%d col=%d text='%s'%n",
        tokenName, t.getLine(), t.getCharPositionInLine(), t.getText());
}
```

Minimaler Java-Code: Text -> Baum

```
var input = CharStreams.fromString(text);  
var lexer = new MiniCalcLexer(input);  
var tokens = new CommonTokenStream(lexer);  
  
var parser = new MiniCalcParser(tokens);  
var tree = parser.prog(); // Wurzelknoten des Baums (Startregel der Grammatik)  
  
IO.println(tree.toStringTree(parser));
```

Der Parse-Tree: Struktur

```
grammar MiniCalc;
```

```
prog  : stmt* EOF ;
```

```
stmt  : ID '=' expr ';' ;
```

```
      | expr ';' ;
```

```
      ;
```

```
expr  : INT ('+' INT)* ;
```

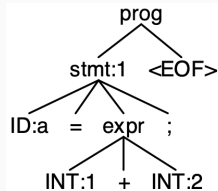
```
ID    : [a-z][a-zA-Z0-9_]* ;
```

```
INT   : [0-9]+ ;
```

```
WS    : [ \t\r\n]+ -> skip ;
```

Eingabe `a = 1 + 2;` liefert:

(prog (stmt a = (expr 1 + 2) ;) <EOF>)



Der Parse-Tree: Klassenhierarchie

```
grammar MiniCalc;
```

```
prog : stmt* EOF ;
```

```
stmt : ID '=' expr ';' |
```

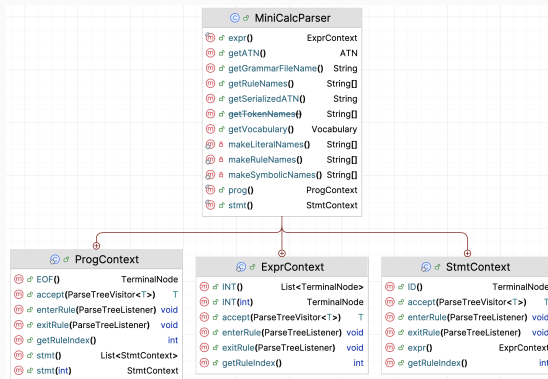
```
      expr ';' ;
```

```
expr : INT ('+' INT)* ;
```

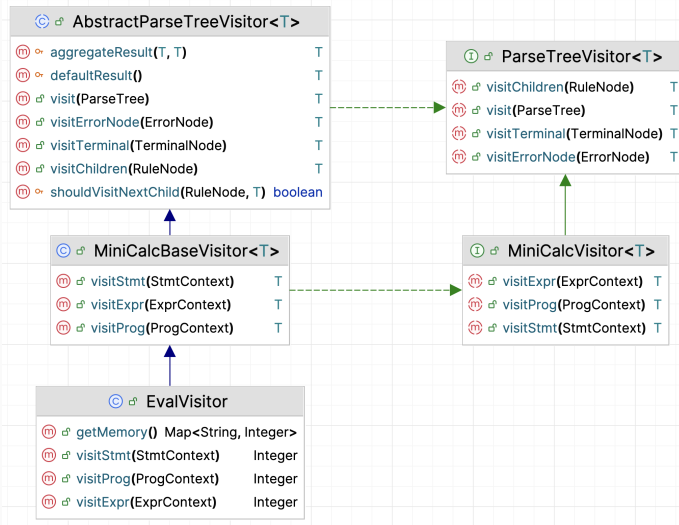
```
ID : [a-z][a-zA-Z0-9_]* ;
```

```
INT : [0-9]+ ;
```

```
WS : [ \t\r\n]+ -> skip ;
```



Traversierung mit Visitor-Pattern



Wrap-Up

- ANTLR als Werkzeug: aus einer gegebenen Grammatik werden Lexer, Parser, Kontextklassen und Visitor-Schnittstellen generiert
- Einbettung in ein Java/Gradle-Projekt:
 - `antlr`-Plugin in `build.gradle`
 - `.g4`-Dateien unter `src/main/antlr/`
 - generierte Sourcen im Build-Ordner
- Üblicher Workflow: Eingabetext -> CharStream -> Lexer -> TokenStream -> Parser -> Parse-Tree
- Struktur des Parse-Baums:
 - jedes Knotenobjekt ist eine `XxxContext`-Klasse
 - Blätter sind Token
 - Whitespaces/Kommentare werden i.d.R. übersprungen/entfernt
- Visitor-Pattern in ANTLR:
 - eigene Klasse von generiertem `FooBaseVisitor<T>` ableiten
 - `visitXxx`-Methoden bei Bedarf überschreiben
 - Baum mit `visitor.visit(tree)` traversieren.



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

